



PDF Download
3528588.3528661.pdf
09 March 2026
Total Citations: 17
Total Downloads: 102

 Latest updates: <https://dl.acm.org/doi/10.1145/3528588.3528661>

SHORT-PAPER

Predicting issue types with seBERT

ALEXANDER TRAUTSCH, The University of Göttingen, Göttingen, Niedersachsen, Germany

STEFFEN HERBOLD, Clausthal University of Technology, Clausthal-Zellerfeld, Niedersachsen, Germany

Open Access Support provided by:

The University of Göttingen

Clausthal University of Technology

Published: 01 February 2023

Citation in BibTeX format

ICSE '22: 44th International Conference
on Software Engineering
May 21, 2022
Pennsylvania, Pittsburgh

Conference Sponsors:
SIGSOFT

Predicting Issue Types with seBERT

Alexander Trautsch
alexander.trautsch@cs.uni-goettingen.de
University of Goettingen
Göttingen, Germany

Steffen Herbold
steffen.herbold@tu-clausthal.de
TU Clausthal
Clausthal-Zellerfeld, Germany

ABSTRACT

Pre-trained transformer models are the current state-of-the-art for natural language models processing. seBERT is such a model, that was developed based on the BERT architecture, but trained from scratch with software engineering data. We fine-tuned this model for the NLBSE challenge for the task of issue type prediction. Our model dominates the baseline fastText for all three issue types in both *recall* and *precision* to achieve an overall *F1-score* of 85.7%, which is an increase of 4.1% over the baseline.

CCS CONCEPTS

• **Software and its engineering** → **Software notations and tools.**

KEYWORDS

natural language processing, seBERT, BERT, issue type prediction

ACM Reference Format:

Alexander Trautsch and Steffen Herbold. 2022. Predicting Issue Types with seBERT. In *The 1st Intl. Workshop on Natural Language-based Software Engineering (NLBSE'22)*, May 21, 2022, Pittsburgh, PA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3528588.3528661>

1 INTRODUCTION

Accurate issue type prediction can support developers and researchers [4]. Within this paper, we show how the pre-trained transformer model seBERT [11] can be used for this purpose as part of the NLBSE 2022 tool competition [6]. The seBERT model is a Natural Language Processing (NLP) model that was trained from scratch for the use within the software engineering domain, based on data from Stack Overflow, Jira and GitHub. seBERT uses the BERT_{LARGE} architecture [2]. In comparison, the original BERT was trained for the general domain and has sometimes problems with understanding words within the software engineering context [11]. We already showed on a different data set that seBERT outperforms BERT and fastText [3] for the identification of bug issues [11]. We now transfer these results to the larger context of issue type prediction as a multi-class problem for the differentiation between the classes *bug*, *enhancement*, and *question*.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to publish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

NLBSE'22, May 21, 2022, Pittsburgh, PA, USA
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9343-0/22/05...\$15.00
<https://doi.org/10.1145/3528588.3528661>

Table 1: Overview of the amount of data for each class in the training and test data.

Class	#Development	#Test
<i>bug</i>	361,238	40,152
<i>enhancement</i>	299,287	33,290
<i>question</i>	62,373	7,076
Total	722,899	80,518

2 ISSUE TYPE PREDICTION WITH SEBERT

We now describe the data we used, our model architecture and training, the performance criteria we use, and the results we achieve. Our code for the experiments is available online.¹ We also provide a live version of the model.²

2.1 Data and Preprocessing

The data we use consists of 722,899 issues collected from GitHub as development data, and 80,518 issues for testing [6–8]. We have the following information for each issue:³

- The label, which is one of the three classes we want to predict, i.e., *bug*, *enhancement*, and *question*.
- The title of the issue.
- The description of the issue, also referred to as body.

We concatenate the title and body of the issue. Then, we replace line breaks with whitespaces and remove multiple subsequent whitespaces. Moreover, we encode the label as an integer, such that *bug* is zero, *enhancement* is one, and *question* is two. Moreover, we split the development data into 80% training data and 20% validation data.

2.2 Model Architecture and Fine-tuning

We use a transfer learning approach based on the fine-tuning of a pre-trained model. This means that we re-use a neural network that was trained for a different purpose. Figure 1 shows an overview of our model which is based on the `BertForSequenceClassification` implementation of HuggingFace.⁴

We use seBERT [11] as foundation for our approach, a pre-trained BERT [2] model which is based on the transformer architecture [10]. In comparison to the BERT model of Google, seBERT was trained from scratch based on data from the Software Engineering (SE) domain. Moreover, seBERT only has an input length of 128 tokens,⁵

¹https://github.com/atrautsch/nlbse2022_replication_kit

²<https://user.informatik.uni-goettingen.de/~trautsch2/nlbse2022/>

³The data also contains the URL of the issue and repository, the time of creation, and the author, but we do not use these fields.

⁴https://huggingface.co/docs/transformers/model_doc/bert#transformers.BertForSequenceClassification

⁵Words are tokenized based on the vocabulary. Words within the vocabulary are used as is, unknown words are represented by subword tokens.

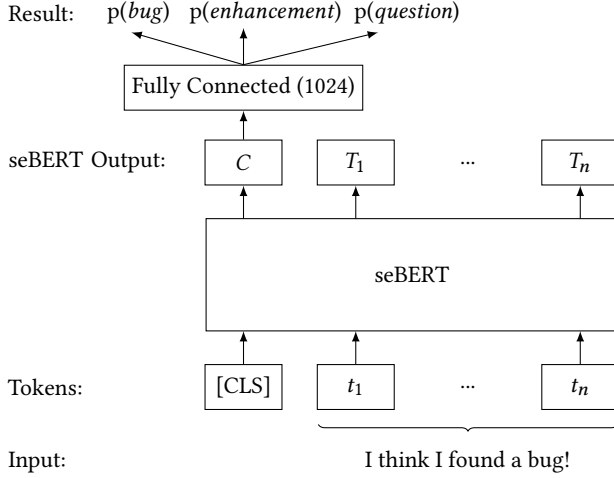


Figure 1: Overview of the architecture of our model.

in comparison to the 512 tokens from the original BERT models. Thus, our seBERT model only reasons about the first 128 tokens, i.e., only the title and the beginning of the issue text. BERT models use the [CLS] token and the output C as placeholder for a classification task. C is a 768 dimensional vector, that we use as input for one fully connected layer with 1024 neurons. The fully connected layer learns a logistic regression over C and computes the probabilities of each class. We achieve this by using the cross-entropy loss [1]. For more details regarding seBERT and BERT we refer the to literature [2, 11].

We initialized our architecture with the weights of the pre-trained seBERT model. We then trained the model for five epochs on the training data. We did not freeze any weights, i.e., we did not just learn the weights of the fully connected layer, but also allowed updating the internal weights of seBERT. As optimizer, we used AdamW [9] with a learning rate of $lr = 5 \cdot 10^{-5}$, $\beta_1 = 0.9$ for the moving average of the gradients, $\beta_2 = 0.999$ for the moving average of the squared gradients, and no weight decay. We validated each epoch on the validation set and use the model with the highest validation accuracy, which was already achieved after the first epoch. No further hyperparameter tuning was applied.

2.3 Performance Criteria

The performance criteria we use are based on tp_c , fn_c , and fp_c , which are defined as follows.

- tp_c are the true positive predictions of the class $c \in C = \{bug, enhancement, question\}$. This means that the class c is predicted correctly.
- fn_c are the false negative predictions of the class c . This means that the class c is missed and is instead predicted as a different class.
- fp_c are the false positive predictions of class c . This means that a different class is wrongly predicted as class c .

Based on these values, we define our performance criteria for each class $recall_c$, $precision_c$, and $F-Score_c$:

Table 2: Performance of seBERT in comparison to the baseline fastText.

		fastText	seBERT	Difference
<i>recall</i>	<i>bug</i>	81.6%	90.6%	+9.0%
	<i>enhancement</i>	84.5%	87.7%	+3.2%
	<i>question</i>	35.0%	48.7%	+13.7%
<i>precision</i>	<i>bug</i>	83.1%	86.6%	+3.5%
	<i>enhancement</i>	81.6%	86.4%	+4.8%
	<i>question</i>	65.2%	73.1%	+7.9%
<i>F-score</i>	<i>bug</i>	83.1%	88.6%	+5.5%
	<i>enhancement</i>	83.1%	87.1%	+4.0%
	<i>question</i>	45.6%	58.4%	+12.8%
<i>micro average</i>		81.6%	85.7%	+4.1%

$$recall_c = \frac{tp_c}{tp_c + fn_c} \quad (1)$$

$$precision_c = \frac{tp_c}{tp_c + fp_c} \quad (2)$$

$$F-Score_c = 2 \cdot \frac{recall_c \cdot precision_c}{recall_c + precision_c} \quad (3)$$

The above criteria measure the performance for the individual classes. We use the *micro average*⁶ of these scores for our overall comparison of the performance:

$$micro\ average = \frac{\sum_{c \in C} tp_c}{\sum_{c \in C} tp_c + fn_c} \quad (4)$$

2.4 Results

Table 2 reports the performance of seBERT on the test data in comparison to the baseline model fastText. Overall, we achieve a *micro average* score of 85.7%, an improvement of 4.1% over fastText. The results show that seBERT is superior to the baseline in all criteria, i.e., we are both better at correctly identifying issues of each class (*recall*), as well as reducing the noise among these predictions (*precision*). This means that the improvement is not at the cost of predictive power for one class, but benefits all classes. The largest improvement of seBERT is for the class *question*, with an increase of 12.8% in *F-score*. However, we note that seBERT still misses over 50% of the questions, i.e., there is still room for improvement. For the classes *bug* and *enhancement*, the performance of seBERT is much stronger, with *F-scores* of 88.6%, respectively 87.1%. However, the difference to fastText is smaller, i.e., the improvement is only by 5.5% for *bug*, respectively 4.0% for *enhancement*.

3 THREATS TO VALIDITY

There is one notable threat to the internal validity of our results, due to the way seBERT was created. The corpus used for pre-training included the Github Issues (including comments) from 2015 to 2019 [11]. The pre-training was self-supervised based on Masked-Language Modelling (MLM) and Next Sentence Prediction (NSP),

⁶Please note that we use the micro average of the *recall* directly, since the micro averages of *recall*, *precision*, and *F-score* are equal. This is because $\sum_{c \in C} fp_c = \sum_{c \in C} fn_c$, which leads to the equality of the micro average of *recall* and *precision*. Since the *F-Score* is their harmonic mean, it follows that is also equal.

i.e., did not include any information about the labels of the issues. However, we cannot rule out the possibility that the issue type prediction works better with data seen during the pre-training, because it is more likely that the language structure is already known by the model. Still, we believe that this threat is low: having seen the data before also increases the risk of memorization instead of generalization, which would actually decrease the performance on the test data.

4 CONCLUSION

We demonstrated the strong performance of the seBERT model for the task of issue type prediction over smaller models like fastText that were not pre-trained on a large corpus of data. While the performance is still not perfect, especially with respect to the detection of questions, results from earlier work demonstrate that humans are also not perfect at this task and there is likely some label noise [4, 5]. Still, there is room for improvement with pre-trained models. seBERT uses only a short input size of 128 tokens, which should be sufficient for most issues [11], but may lead to missing vital information later in the issue for longer issues. Thus, models that are able to consider longer inputs may yield further improvements. For example, the recent Big Bird model is able to consider inputs with up to 4096 tokens [12]. Moreover, we did not conduct extensive hyperparameter tuning to find the best possible training configuration (e.g., best learning rate) for seBERT, which means we possibly underestimate the capabilities of seBERT.

REFERENCES

- [1] Pieter-Tjerk De Boer, Dirk P Kroese, Shie Mannor, and Reuven Y Rubinstein. 2005. A tutorial on the cross-entropy method. *Annals of operations research* 134, 1 (2005), 19–67.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, Minneapolis, Minnesota, 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
- [3] Facebook AI Research. 2019. fastText - Library for efficient text classification and representation learning. <https://fasttext.cc/>. [accessed 14-November-2019].
- [4] Steffen Herbold, Alexander Trautsch, and Fabian Trautsch. 2020. On the feasibility of automated prediction of bug and non-bug issues. *Empirical Software Engineering* 25, 6 (Sept. 2020), 5333–5369. <https://doi.org/10.1007/s10664-020-09885-w>
- [5] Kim Herzig, Sascha Just, and Andreas Zeller. 2013. It's Not a Bug, It's a Feature: How Misclassification Impacts Bug Prediction. In *Proceedings of the 2013 International Conference on Software Engineering (ICSE)*. IEEE.
- [6] Rafael Kallis, Oscar Chaparro, Andrea Di Sorbo, and Sebastiano Panichella. 2022. NLBSE'22 Tool Competition. In *Proceedings of The 1st International Workshop on Natural Language-based Software Engineering (NLBSE'22)*.
- [7] Rafael Kallis, Andrea Di Sorbo, Gerardo Canfora, and Sebastiano Panichella. 2019. Ticket Tagger: Machine Learning Driven Issue Classification. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 406–409. <https://doi.org/10.1109/ICSME.2019.00070>
- [8] Rafael Kallis, Andrea Di Sorbo, Gerardo Canfora, and Sebastiano Panichella. 2021. Predicting issue types on GitHub. *Science of Computer Programming* 205 (2021), 102598. <https://doi.org/10.1016/j.scico.2020.102598>
- [9] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=Bkg6RiCqY7>
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. *CoRR* abs/1706.03762 (2017). arXiv:1706.03762 <http://arxiv.org/abs/1706.03762>
- [11] Julian von der Mosel, Alexander Trautsch, and Steffen Herbold. 2021. On the validity of pre-trained transformers for natural language processing in the software engineering domain. arXiv:2109.04738 [cs.SE]
- [12] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big Bird: Transformers for Longer Sequences. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 17283–17297. <https://proceedings.neurips.cc/paper/2020/file/c8512d142a2d849725f31a9a7a361ab9-Paper.pdf>